



DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

First Assignment

Equivalent representations of orientation matrices

Author:

Marina Pedemonte
Francesco Sisani

Professors:

Enrico Simetti
Giorgio Cannata

Student ID:

S5622904
S8571674

Tutors:

Luca Tarasi
Simone Borelli

December 1, 2025

Contents

1	Assignment description	3
1.1	Exercise 1 - Angle-Axis to Rotation Matrix	3
1.2	Exercise 2 - Rotation Matrix to Angle-Axis	3
1.3	Exercise 3 - Euler Angles to Rotation Matrix	4
1.4	Exercise 4 - Rotation Matrix to Euler Angles	4
1.5	Exercise 5 - Rotation Matrix to Angle-Axis using Eigenvectors	4
1.6	Exercise 6 - Frame tree	5
2	Exercise 1	6
2.1	Q1.1	6
2.2	Q1.2	6
2.3	Q1.3	6
2.4	Q1.4	6
3	Exercise 2	7
3.1	Q2.1	7
3.2	Q2.2	8
3.3	Q2.3	8
3.4	Q2.4	8
3.5	Q2.5	9
3.6	Q2.6	9
4	Exercise 3	9
4.1	Q3.1	9
4.2	Q3.2	9
4.3	Q3.3	10
4.4	Q3.4	10
4.5	Q3.5	10
5	Exercise 4	10
5.1	Q4.1	10
5.2	Q4.2	11
5.3	Q4.3	11
5.4	Q4.4	11
6	Exercise 5	11
6.1	Q5.1.1	11
6.2	Q5.1.2	12
7	Exercise 6	12
7.1	Q6.1	12

Mathematical expression	Definition	MATLAB expression
$\langle w \rangle$	World Coordinate Frame	w
${}^a_b R$	Rotation matrix of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aRb
${}^a_b T$	Transformation matrix of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aTb

Table 1: Nomenclature Table

1 Assignment description

The first assignment of Modelling and Control of Manipulators focuses on the geometric fundamentals and algorithmic tools underlying any robotics application. The concepts of transformation matrix, orientation matrix and the equivalent representations of orientation matrices (Equivalent angle-axis representation and Euler Angles) will be reviewed.

The first assignment is **mandatory** and consists of 5 different exercises. You are asked to:

- Download the .zip file called MCM-LAB1 from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling the predefined files called "main.m", "AngleAxisToRot.m", "RotToAngleAxis.m", "YPRToRot.m", "RotToYPR.m" and "IsRotationMatrix.m".
- Write a report motivating the answers for each exercise, following the predefined format on this document.
- The usage of built-in MATLAB functions is strictly forbidden except for basic mathematical operations such as *det*, *eig*.

1.1 Exercise 1 - Angle-Axis to Rotation Matrix

A particularly interesting minimal representation of 3D rotation matrices is the so-called angle-axis representation, where a rotation is represented by the axis of rotation $\mathbf{h}$ and the angle θ . Any rotation matrix can be represented by its equivalent angle-axis representation by applying the Rodrigues Formula.

Q1.1 Given an angle-axis pair $(\mathbf{h}, \theta)$, implement on MATLAB the Rodrigues formula, computing the equivalent rotation matrix, **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } R = \text{AngleAxisToRot}(h, \theta)$$

Then test it for the following cases and briefly comment the results obtained:

- **Q1.2** $\mathbf{h} = [1, 0, 0]^T$ and $\theta = 90^\circ$
- **Q1.3** $\mathbf{h} = [0, 0, 1]^T$ and $\theta = \pi/3$
- **Q1.4** $\rho = [-\pi/3, -\pi/6, \pi/3];$

1.2 Exercise 2 - Rotation Matrix to Angle-Axis

Given a rotation matrix R , the problem of finding the corresponding angle-axis representation $(\mathbf{h}, \theta)$ is called the Inverse Equivalent Angle-Axis Problem.

Q2.1 Given a rotation matrix R , implement on MATLAB the Equivalent Angle-Axis equations **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } [h, \theta] = \text{RotToAngleAxis}(R)$$

You **MUST** check that the input is a valid rotation matrix by filling in and utilizing the function *IsRotationMatrix(R)*.

Hint: utilize a suitable tolerance (e.g. 10^{-3}) to check the properties of the matrices.

Test it for the following cases and briefly comment the results obtained:

- **Q2.2** $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$
- **Q2.3** $R = \begin{pmatrix} 0.5 & -\sqrt{3}/2 & 0 \\ \sqrt{3}/2 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- **Q2.4** $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- **Q2.5** $R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

• **Q2.6**
$$R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

1.3 Exercise 3 - Euler Angles to Rotation Matrix

Any orientation matrix can be expressed in terms of three elementary rotations in sequence. Consider the Yaw Pitch Roll (YPR) representation, where the sequence of the rotation axes is Z-Y-X.

Q3.1 Given a triplet of YPR angles (ψ, θ, ϕ) , compute the equivalent rotation matrix representation **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } R = \text{YPRToRot}(\text{psi}, \text{theta}, \text{phi})$$

Then test it for the following cases and briefly comment the results obtained:

- **Q3.2** $\psi = \theta = 0, \phi = \pi/2$
- **Q3.3** $\phi = \theta = 0, \psi = 60^\circ$
- **Q3.4** $\psi = \pi/3, \theta = \pi/2, \phi = \pi/4$
- **Q3.5** $\psi = 0, \theta = \pi/2, \phi = -\pi/12$

1.4 Exercise 4 - Rotation Matrix to Euler Angles

Given a rotation matrix R , it is possible to compute an equivalent triplet of YPR angles (ψ, θ, ϕ) , provided that the configuration is not singular (that is, $\cos \theta \neq 0$).

Q4.1 Given a rotation matrix R , implement in MATLAB the equivalent YPR angles, **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } [\text{psi}, \text{theta}, \text{phi}] = \text{RotToYPR}(R)$$

You **MUST** check that the input is a valid rotation matrix by filling in and utilizing the function *IsRotationMatrix(R)*. **Hint:** utilize a suitable tolerance (e.g. 10^{-3}) to check the properties of the matrices.

Test it for the following cases and briefly comment the results obtained:

- **Q4.2**
$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$
- **Q4.3**
$$R = \begin{pmatrix} \frac{1}{2} & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix}$$
- **Q4.4**
$$R = \begin{pmatrix} 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0.5 & \frac{\sqrt{2}\sqrt{3}}{4} & \frac{\sqrt{2}\sqrt{3}}{4} \\ -\frac{\sqrt{3}}{2} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \end{pmatrix}$$

1.5 Exercise 5 - Rotation Matrix to Angle-Axis using Eigenvectors

Consider a rotation matrix R and the corresponding angle-axis representation $(\mathbf{h}, \theta)$. Then, $R\mathbf{h} = \mathbf{h}$, which can be verified by computing $R\mathbf{h}$ with the Rodrigues Formula and observing that the terms containing $\mathbf{h} \times \mathbf{h}$ vanish. Therefore, $\mathbf{h}$ is an eigenvector of R corresponding to the eigenvalue +1.

Given the following rotation matrices, compute the corresponding angle-axis representation with two different methods. In the first method, compute $(\mathbf{h}, \theta)$ using the function *RotToAngleAxis* implemented before. In the second one, compute $\mathbf{h}$ as the eigenvector of R corresponding to eigenvalue +1, and then use the function *RotToAngleAxis* to compute θ . Compare and briefly discuss the results.

• **Q5.1**
$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

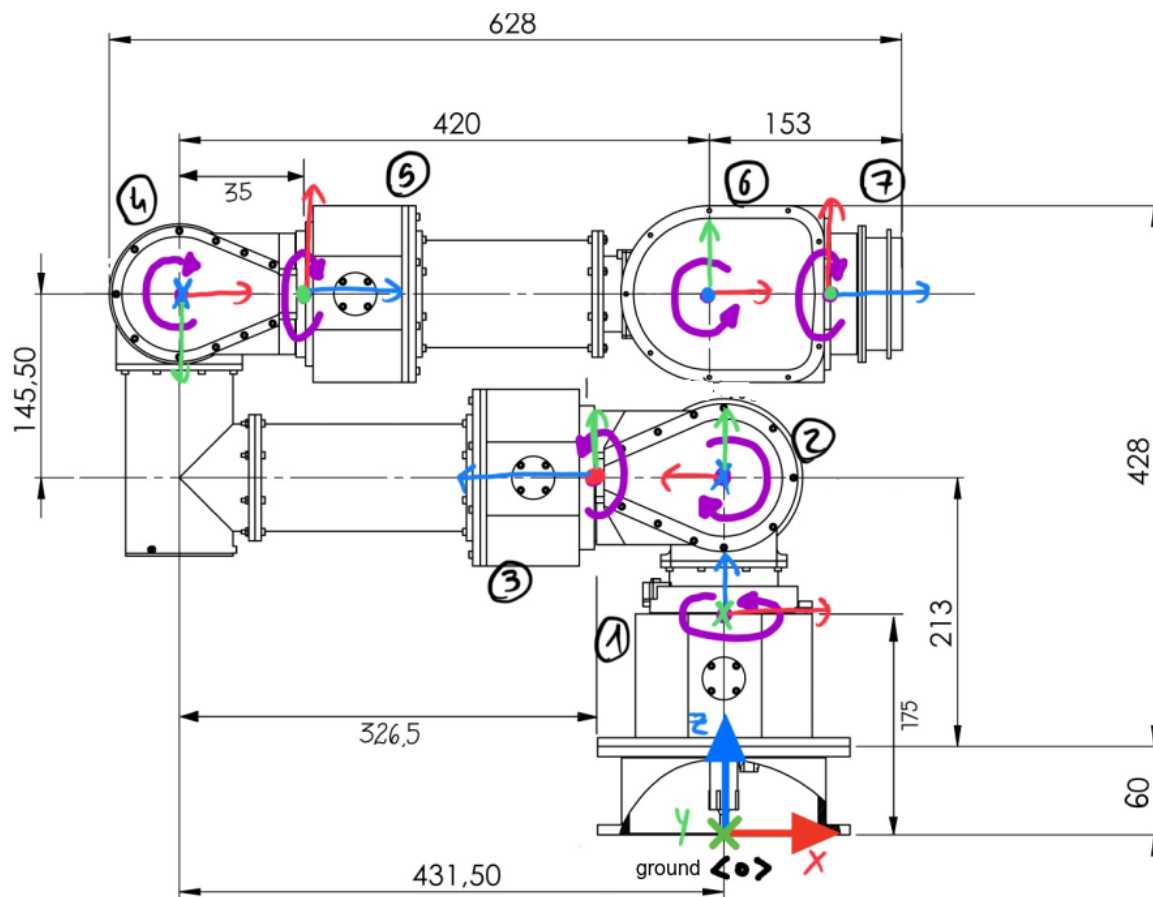


Figure 1: Exercise 6 frames. The unit of measurement is millimeters.

• Q5.2
$$R = \frac{1}{9} \begin{pmatrix} 4 & -4 & -7 \\ 8 & 1 & 4 \\ -1 & -8 & 4 \end{pmatrix}$$

1.6 Exercise 6 - Frame tree

Figure 1 shows the frame tree for the 7 joints of the Franka robot. With reference to the figure, use the geometric definition of the transformation matrix to compute by hand the following matrices.

- Q6.1 0_1T
- Q6.2 1_2T
- Q6.3 2_3T
- Q6.4 3_4T
- Q6.5 4_5T
- Q6.6 5_6T
- Q6.7 6_6T
- Q6.8 7_6T

You **MUST** compute the matrices **WITHOUT** using mathematical software.

2 Exercise 1

2.1 Q1.1

The function `AngleAxisToRot` has the objective of finding the rotation matrix relative to given angle-axis parameters such that:

$$R = e^{\theta[\underline{h}x]} \quad (1)$$

where $\theta \in \mathbb{R}$, $\underline{h} \in \mathbb{R}^3$, $\|\underline{h}\| = 1$.

To obtain θ and $\underline{h}$, respectively defined as a scalar rotation angle and as a unitary vector, we implemented another form of the relation defined in equation [1], i.e. the Rodrigues formula:

$$R = I + (\sin \theta)[\underline{h}x] + (1 - \cos \theta)[\underline{h}x]^2 \quad (2)$$

This formula is obtained by demonstrating the surjectivity of the mapping $e^{\theta[\underline{h}x]} \rightarrow SO(3)$, or:

$$\forall R \in SO(3) \exists (n, \theta) s.t. R = e^{\theta[nx]} \quad (3)$$

with θ and n defined with the above conditions.

In our matlab code the function `AngleAxisToRot` outputs the rotation matrix by, given the parameters, computing the identity matrix in $\mathbb{R}^3$, calculating the vector operator $[\underline{h}x]$ or *h-cross*, defined as follows:

$$[\underline{h}x] = \begin{pmatrix} 0 & -h_z & h_y \\ h_z & 0 & -h_x \\ -h_y & h_x & 0 \end{pmatrix}$$

and then using both of them in the Rodrigues formula together with the angle-axis parameters.

To test our function we used the provided angle axis parameters. In the following sections we show the result obtained.

2.2 Q1.2

Test parameters: $\mathbf{h} = [1, 0, 0]^T$ and $\theta = 90^\circ$

We used radians to describe the value of the angular rotation so $\theta = \pi/2$

The result obtained is the following:

$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

As discussed earlier, since θ denotes the rotation about the fixed axis $\underline{h}$, the matrix R represents a 90° rotation about the fixed *x-axis*.

2.3 Q1.3

Test parameters $\mathbf{h} = [0, 0, 1]^T$ and $\theta = \pi/3$

The result obtained is the following:

$$R = \begin{pmatrix} 0.5 & -0.866 & 0 \\ 0.866 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Also in this case, there is consistency: we perform a rotation of 60° about the *z-axis* that remains unchanged.

2.4 Q1.4

Test parameters: $\rho = [-\pi/3, -\pi/6, \pi/3]$;

In this case, we are provided with a parameter ρ that is called *rotation vector*. It is defined as:

$$\rho = (\theta \underline{h}) \in \mathbb{R}^{3 \times 1}. \quad (4)$$

In order to obtain the angle axis parameters needed, we exploited our knowledge on the $\underline{h}$ vector, which has a unitary norm. So we divided ρ for its norm and obtained $\underline{h}$:

$$\|\rho\| = \sqrt{\frac{\pi^2}{9} + \frac{\pi^2}{9} + \frac{\pi^2}{36}} = \frac{\pi}{2} = \theta$$

$$\frac{\rho}{\|\rho\|} = \begin{pmatrix} -\frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \end{pmatrix} = \underline{h}$$

Note that we arbitrary choose the value of θ as positive and computed $\underline{h}$. We could have used the negative value and the computation of $\underline{h}$ would have provided the same result with all the signs inverted. In both cases we obtain the following result:

$$R = \begin{pmatrix} 0.4444 & -0.4444 & -0.7778 \\ 0.8889 & 0.1111 & 0.4444 \\ -0.1111 & -0.8889 & 0.4444 \end{pmatrix}$$

3 Exercise 2

3.1 Q2.1

The function `RotToAngleAxis` aims to resolve the Inverse Equivalent Angle-Axis problem, which is used to find the couple of angle axis parameters $[\underline{h}, \theta]$ given a certain rotation matrix. The first step we focused on was to actually check if the given test matrices were rotation matrices, and this was done with the building of another function called `isRotationMatrix`. This function checks the validity of a given matrix R by verifying if it is a valid member of the $SO(3)$ group, or special orthogonal group; a matrix belongs in $SO(3)$ if both the properties below are satisfied:

- $RR^T = I$;
- $|\det(R)| = 1$;

The code we implemented for `isRotationMatrix` initializes a boolean variable called `isRotationMatrix` and set to false and an integer counter set to 0. We then proceed to check the first property of the $SO(3)$ group with a for loop that, given a new matrix R_p that stores the product RR^T , iterates through its diagonal to check if R_p is the identity matrix in $\mathbb{R}^3$.

We must specify that the property checks have been done with a tolerance of 10^{-3} , and the iteration on R_p is no exception.

Given this, to check the first property, we have defined a for loop such that, while iterating through the diagonal, if the operation

$$RR^T(i, i) >= 10^{-3} \quad (5)$$

is satisfied, the loop breaks and does not iterate further, whereas it adds 1 to the counter in the opposite outcome. We chose this implementation because, given that an identity matrix's only elements are ones on the diagonal, if only one element of R_p 's diagonal is not the same, then we do not need to iterate any further.

We then proceed to check the second property of $SO(3)$, and we do this through an if-else block that sets `isRotationMatrix` to true only if both of the following statements are satisfied:

$$|\det(R) - 1| < 10^{-3} \quad (6)$$

and the counter is equal to 3; otherwise, it does not do anything; this is because the counter has already checked if the product R_p is the identity, and given the previous reasoning, this can only happen if the counter is equal to 3.

This check is then implemented as the first line of code in `RotToAngleAxis` to verify whether it is worthwhile to experiment on the given matrix. If the check returns false, the code will throw an error and display the message "Error: Input matrix R is not a valid rotation matrix".

After ensuring that R is a rotation matrix, we want to effectively calculate the angle-axis parameters θ and $\underline{h}$. Firstly we calculate θ by using the formula:

$$\theta = \arccos\left(\frac{\text{trace}(R) - 1}{2}\right) \quad (7)$$

and after that we check its different output possibilities in order to calculate $\underline{h}$

- if $\theta = 0$ then $\underline{h}$ is a completely arbitrary vector; in our case, we chose the vector $\underline{h} = [1, 0, 0]^T$

- if $\theta = \pi$ we initialize $\underline{h}$ as a 3×1 null vector and then we use a for loop such that $\underline{h}$ is assigned values with the equation below:

$$h(i) = \sqrt{\frac{R(i,i) + 1}{2}}, i \in 1, 2, 3 \quad (8)$$

We then use another for loop to choose the first possible index in which the following inequality is satisfied:

$$|\sqrt{\frac{R(i,i) + 1}{2}}| > 0 \quad (9)$$

and store it in a variable named *chIndex*.

We then use the following expression inside of another for loop to compute the elements of $\underline{h}$ that do not have index *chIndex*:

$$\text{sgn}(h(\text{chIndex}))\text{sgn}(R(\text{chIndex}, j))\sqrt{\frac{R(j,j) + 1}{2}}, j \in 1, 2, 3 \setminus \text{chindex} \quad (10)$$

and the result is our $\underline{h}$ vector.

The $\text{sgn}(x)$ function used in the previous expression is implemented in the same file as the function and is defined as:

- 1 if $x > 0$;
- -1 if $x < 0$;
- 0 otherwise.

- if $\theta \neq 0, \theta \neq \pi$ we obtain $\underline{h}$ as:

$$\underline{h} = \frac{1}{\sin \theta} \text{vex}\left(\frac{R - R^T}{2}\right) \quad (11)$$

where *vex* is an operator that, given a skew-symmetric matrix *S*, returns its original vector in the form $v = [S(3,2), S(1,3), S(2,1)]$. This operator is also implemented as a new function inside of the same file used for its call.

To test our function we provided matrices. In the following sections we show results both from the *isRotationMatrix* function and eventual angle-axis couples.

3.2 Q2.2

Given matrix: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$

Results as a rotation matrix, angle-axis parameters: $\underline{h} = [1, 0, 0]^T, \theta = \frac{\pi}{2}$

This is the same matrix seen in Q1.2, and thus we see consistency between the two cases in representing a 90° rotation about the x-axis.

3.3 Q2.3

Given matrix: $\begin{pmatrix} 0.5 & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Results as a rotation matrix, angle-axis parameters: $\underline{h} = [0, 0, 1]^T, \theta = \frac{\pi}{3}$

This case is also consistent with its inverse, seen in Q1.3, that represents a 60° rotation around the z-axis.

3.4 Q2.4

Given matrix: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Results as a rotation matrix, angle-axis parameters: $\underline{h} = [1, 0, 0]^T, \theta = 0$

The given matrix is the identity, so we know from the start that it is a rotation matrix. Knowing this, we can also say that any vector $\underline{h}$ can be an angle-axis parameter.

3.5 Q2.5

Given matrix: $\begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Results as a rotation matrix, angle-axis parameters: $\underline{h} = [0, 0, 1]^T, \theta = \pi$

The given matrix has a striking resemblance with the identity matrix, and it results as a rotation matrix because of its determinant, which becomes 1 because of the two negative elements in the diagonal. It represents a 180° rotation on the z-axis.

3.6 Q2.6

Given matrix: $\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

This is not a rotation matrix because its determinant is -1 and therefore it breaks one of the $SO(3)$ group's properties.

4 Exercise 3

4.1 Q3.1

The function we created in Matlab to implement this task is called `YPRToRot` and it has the objective of transforming a triplet of Euler Angles into a Rotation matrix. This function exploits the knowledge about Euler Angles that can represent elementary rotations around elementary axis:

YAW → rotation of the angle ψ around Z-axis

PITCH → rotation of the angle θ around Y-axis

ROLL → rotation of the angle ϕ around X-axis

Rotations can be intrinsic or extrinsic, meaning that the axis respectively, rotate with the frame or stay fixed. In our case we consider intrinsic rotation and this allows us to write the rotation matrix R as the product of singular elementary rotations:

$$R = R_z(\psi) R_y(\theta) R_x(\phi)$$

Each of them can be expressed as follows, which is what we implemented in Matlab:

$$R_z(\psi) = \begin{pmatrix} \cos(\psi) & -\sin(\psi) & 0 \\ \sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$R_y(\theta) = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{pmatrix}$$

$$R_x(\phi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi) & \cos(\phi) \end{pmatrix}$$

It would also have been possible to combine all the elementary rotations into one single matrix, obtaining the same results.

To test our function we used the provided triplets of Euler angles. In the following sections we show the result obtained.

4.2 Q3.2

Test values:

$$\psi = \theta = 0, \quad \phi = \frac{\pi}{2}$$

The result obtained is the following:

$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

This matches the outcome obtained in 2.2, where we were asked to perform a 90-degree rotation around the x-axis. This confirms the consistency of our results.

4.3 Q3.3

Test values:

$$\phi = \theta = 0, \quad \psi = 60^\circ$$

The result obtained is the following:

$$R = \begin{pmatrix} 0.5 & -0.866 & 0 \\ 0.866 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Also in this case we show consistency, the matrix R is the same measured in 2.3

4.4 Q3.4

Test values:

$$\psi = \frac{\pi}{3}, \quad \theta = \frac{\pi}{2}, \quad \phi = \frac{\pi}{4}$$

The result obtained is the following:

$$R = \begin{pmatrix} 0 & -0.258 & 0.9659 \\ 0 & 0.9659 & 0.258 \\ -1 & 0 & 0 \end{pmatrix}$$

4.5 Q3.5

Test values:

$$\psi = 0, \quad \theta = \frac{\pi}{2}, \quad \phi = -\frac{\pi}{12}$$

The result obtained is the following:

$$R = \begin{pmatrix} 0 & -0.258 & 0.9659 \\ 0 & 0.9659 & 0.258 \\ -1 & 0 & 0 \end{pmatrix}$$

As shown in the image below, the rotation matrices R3 and R4 are the same even though the corresponding Euler angles are different. This happens because the two YPR triplets apply the same rotations around the axes; in both cases we have a rotation of 90° around the y axis, while ψ and ϕ are different, but apply the same rotations, differing only by how these rotations are applied.

5 Exercise 4

5.1 Q4.1

The function `RotToYPR` aims to find the solution to the previous problem's inverse, which means to find an equivalent triplet of angles (ψ, θ, ϕ) given a rotation matrix.

As a first measure, we use again the function `isRotationMatrix` to check if the given matrix is a rotation matrix; this is necessary because, as already said, yaw, pitch and roll are defined as a rotation sequence around fixed axes, and therefore cannot be defined without a rotation matrix.

After the check, we calculate θ by following this equation:

$$\theta = \text{atan2}(R(3,1), \sqrt{R(1,1)^2 + R(2,1)^2}) \quad (12)$$

We then check the resulting theta:

- if $\theta \neq 0$ we can compute ψ and ϕ without any issues since it is possible to obtain a unique triplet:

$$\psi = \text{atan2}(R(2,1), R(1,1)) \quad (13)$$

$$\phi = \text{atan2}(R(3,2), R(3,3)) \quad (14)$$

- if $\theta = 0$ we can not calculate a definite YPR triplet since their representation is singular, and therefore it is not possible to define a unique YPR configuration. The program will thus throw an error if $\theta = 0$.

After further testing, especially during the calculation of matrices in section Q6.1, we found out that this implementation returned ambiguous results when handling negative values in the YPR triplet, so we expanded the code implementation with a tolerance check and a normalization procedure in order to maintain consistent analysis on every possible case.

To test this function we provide to-be-checked rotation matrices. In the following sections we show the respective results.

5.2 Q4.2

Given matrix: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$

YPR triplet: $\psi = 0, \theta = 0, \phi = \frac{\pi}{2}$

The given matrix is the same as the one seen in Q1.2 and Q2.2, and this case is indeed consistent with the previous experiments since we only find $\phi = \frac{\pi}{2}$ and, given that ψ , θ and ϕ respectively represent extrinsic rotations on the z, y and x axes, we see that $\phi = \frac{\pi}{2}$ represents a 90° rotation about the z-axis, as already demonstrated.

5.3 Q4.3

Given matrix: $\begin{pmatrix} \frac{1}{2} & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix}$

YPR triplet: $\psi = \frac{\pi}{3}, \theta = 0, \phi = 0$

Even in this case the matrix coincides with a previously seen one, specifically the one from sections Q1.3 and Q2.3. We only find $\psi = \frac{\pi}{3}$, which, according to the definition of yaw, pitch and roll, represents a 60° rotation about the x-axis, as previously demonstrated.

5.4 Q4.4

Given matrix: $\begin{pmatrix} 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0.5 & \frac{\sqrt{2}\sqrt{3}}{4} & \frac{\sqrt{2}\sqrt{3}}{4} \\ -\frac{\sqrt{3}}{2} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \end{pmatrix}$

YPR triplet: $\psi = \frac{\pi}{2}, \theta = \frac{\pi}{3}, \phi = \frac{\pi}{4}$

This matrix involves extrinsic rotations on all the axis, and it can be seen both by way, pitch and roll being all not zero and by the fact that, in the matrix itself, no axis maintains the identity structure.

6 Exercise 5

In this exercise we were asked to develop two methods for computing the angle-axis parameters starting from a rotation matrix. The first method is based on the Rodrigues Formula that we implemented on MATLAB with the function `RotToAngleAxis` explained in 3.1. This allows us to compute the parameters θ and $\underline{h}$.

With the second method we exploited another property of $\underline{h}$ in order to get its value. It can be demonstrated that $\underline{h}$ is an eigenvector of R corresponding to the eigenvalue 1. We implemented this calculations on MATLAB and obtained the following results.

6.1 Q5.1.1

Provided test matrix:

$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

In this case with the first and the second method we obtain exactly the same result $\underline{h} = [1, 0, 0]^T, \theta = \frac{\pi}{2}$.

6.2 Q5.1.2

Provided test matrix:

$$R = \frac{1}{9} \begin{pmatrix} 4 & -4 & -7 \\ 8 & 1 & 4 \\ -1 & -8 & 4 \end{pmatrix}$$

In this case, instead, we obtain two *different* results using method one and method two. The output obtained on MATLAB is the following: $\underline{h}_2 = [0.6667, 0.3333, -0.6667]^T$, $\underline{h}_1 = [-0.6667, -0.3333, 0.6667]^T$, $\theta_1 = \frac{\pi}{2}$. Results differ just for opposite signs in vector $\underline{h}$.

This is due to the fact that when we compute an eigenvector, we solve the following equation which returns a unit vector defined up to its sign.

$$R\underline{h} = \underline{h},$$

We can prove that if $\underline{h}$ is an eigenvector associated with the eigenvalue 1, then $-\underline{h}$ is also a valid eigenvector, since

$$R(-\underline{h}) = -(R\underline{h}) = -\underline{h}.$$

Therefore, the eigenvector corresponding to the eigenvalue 1 does not have a unique direction, both $\underline{h}$ and $-\underline{h}$ are valid solutions.

This generates ambiguity, since a rotation matrix can be equivalently represented by the pairs $(\theta, \underline{h})$ and $(-\theta, -\underline{h})$, which describe the same physical rotation. However, keeping θ fixed while changing the sign of $\underline{h}$ results in a different rotations. Moreover, when θ is computed from the trace of the matrix,

$$\theta = \arccos\left(\frac{\text{tr}(R) - 1}{2}\right),$$

the result is always non-negative because $\arccos(\cdot)$ returns values in the range $[0, \pi]$. As a consequence, this formula provides only the magnitude of the rotation and does not allow to infer its direction. To determine the correct sense of rotation, the sign of $\sin \theta$ must be deduced from this formula:

$$\sin \theta \underline{h} = \text{vex}\left(\frac{R - R^T}{2}\right),$$

which ensures that the obtained rotation axis is consistent with the true orientation encoded in the matrix R . In conclusion, for our case, if we consider the vector $\underline{h}_2 = [0.6667, 0.3333, -0.6667]^T$ obtained with the method of eigenvectors, and we use the formula shown before to compute θ we obtain: $\theta_2 = -\frac{\pi}{2}$ that is consistent with all the discussion.

7 Exercise 6

7.1 Q6.1

$${}^0_1T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 175 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

This transformation matrix shows a translation of 175 mm along the z-axis of position 0, due to the fact that the rotation matrix contained in it is the identity.

$${}^1_2T = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 98 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

After testing with the previously defined function `RotToYPR`, we find that the transformation 1_2T is composed by a rotation around position frame 1's z-axis (180°) and x-axis (90°) and a translation of 95 mm on position 1's z-axis.

$${}^2_3T = \begin{pmatrix} 0 & 0 & 1 & 105 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

We can see directly from the rotation matrix that the transformation 2_3T is composed by a rotation around frame 2's y-axis, since the y-axis is not changed. Moreover, by further testing we see that this rotation is of 90° around that axis. Position frame 3 is also derived from a translation of frame 2 on its x-axis of 105 mm.

$${}^3_4T = \begin{pmatrix} 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 145.5 \\ -1 & 0 & 0 & 326.5 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

In this case testing provides that what contributes to 3_4T is a rotation of 90° around the y-axis and 180° around the z-axis. However, in this case, we have the first and only instance of compound translation, since position 4 references position 3 with a translation of 145.5 mm on the y-axis and a translation of 326.5 mm on the z-axis.

$${}^4_5T = \begin{pmatrix} 0 & 0 & 1 & 35 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

This transformation matrix is formed by a compound rotation of frame 4 around its z and x axes, both of -90° , and a translation of 35 mm on frame 4's x-axis.

$${}^5_6T = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 385 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

This transformation matrix is formed by a rotation of -90° both on z and y-axis of frame 5, and a translation of 385 mm on frame 5's y-axis.

$${}^6_6T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

This transformation matrix is an identity, since represents the system 6 compared to itself. It therefore does not rotate, and we know this by seeing that the contained rotation matrix is the identity, nor does it translate, because the vector representing the position of the origin of 6 compared to itself is the null vector of dimension 3.

$${}^7_6T = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & -153 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

This transformation matrix differs from the others because in the previous cases we were the referencing frame was the next position in line, whereas in this case the referencing frame is the previous one. The rotation matrix is the same as 5_6T and thus we have a rotation of -90° around both the z and x axes, while frame 7 translates 153 mm on the opposite direction of its z-axis.